Homework Set 6

The Finite Element Method

Revision: 08–Apr–2025

Each of the following problems begins with the code given in

- Julia: `https://github.com/gu-meng451/2025examples/tree/main/fem`
- Python: `https://github.com/gu-meng451/2025examples/tree/main/fem-python`

6.1 Quadratic elements for the 1D bar

Add 3-node quadratic elements to the 1D bar problem. Change the loading to be for a bar that is at equilibrium in a spinning frame of reference. The boundary conditions are that the left end ($x = 0$) is fixed and the right end ($x = L$) is free. Show that the forcing is

$$f = \rho A \Omega^2 x \tag{6.1}$$

where ρ is the density and Ω is the angular velocity. Derive the analytical solution for the displacements and stresses. Compare to the linear element results (both for displacements and for stress). What would need to be done in order to match the results exactly?

6.2 2D conduction

Setup and solve the same conduction problem from HW 3.1. Compare the results with HW 3. Bonus: Implement 9-node quadratic elements and see how the results compare as the mesh is varied.

6.3 Triangles

1. Implement 3 node triangle shape functions (and their derivatives) in a similar coding-pattern to the 4-node quadrilateral elements.
2. Construct appropriate integration routines. The rules of Witherden and Vincent (2015) are high quality and the weights and abscissae are available from the supplemental materials on the publisher's site. Take note that the domain that these are defined in is different from the $[0, 1]$ triangles found in most other places. Build tests to ensure that you're integrations are what you think they are.
3. Setup and solve the same conduction problem from HW 3.1. Compare the results with HW 3.

6.4 2D conduction: boundary conditions

Formulate and implement a convective boundary condition. Test it on a known problem and determine the accuracy.

6.5 Vibrations and the eigenvalue problem

1. Formulate and implement building the mass matrix for the 1D bar problem.

2. Consider a uniform 1D string that is fixed at $x = 0$ and $x = L$. Compute the natural frequencies and modeshapes for the first 4 modes of vibration. Plot the results. Remember that *after* essential boundary conditions are applied the eigenvalue problem is

$$\left(-\omega^2 \mathbf{M} + \mathbf{K}\right) \boldsymbol{v} = \mathbf{0} \tag{6.2}$$

   where ω is a natural frequency and $\boldsymbol{v}$ is its associated modeshape.

6.6 2D Elasto-statics

Implement the 2D plane-stress/plane-strain problem on quadrilaterals. Consider boundary conditions of distributed loading. Develop a simple test case and verify your program.

6.7 3D Elasto-statics

Implement the 3D linear elastic problem on hexahedra. Consider boundary conditions of distributed loading. Develop a simple test case and verify your program.

6.8 Elasto-dynamics

Using one of the 1D, 2D, or 3D codes above, implement the elasto-dynamic problem as a time-stepping problem.

1. Implement the Newmark-β time-stepping method (Newmark, 1959) using a consistent mass matrix. (Alternatively, you can implement the Generalized-α method of Chung and Hulbert (1993) which can be reduced to several other schemes including the classical Newmark-β. It is slightly more complicated, but it is a superior scheme for controlling high-frequency dissipation.)

2. Develop a simple test case and explore the results as the parameters are changed.

6.9 Misc other ideas to extend the code

1. Implement a 2D truss code. Demonstrate it on a simple problem, and visualize the results.

2. As the number of degrees of freedom increases, the size of the system matrices can become large. Sparse matrix techniques can be used to reduce the memory footprint and speed up the solution. In particular, the `scipy.sparse` package in Python and the `SparseArrays.jl` package in Julia can be used to store the system matrices. Modify the assembly routines to build sparse matrices directly. It is recommended to store the matrices in the *CSC* (Compressed Sparse Column) format as this typically has better performance in LU related solvers.

3. For larger meshes you may want to use Gmsh to build the mesh. To import the meshes into your code, consider using the Python package `meshio`. If you're using Julia, you can call Python packages using PyCall.jl or `PythonCall.jl`.

4. Visualization can be a complicated task, and specialized tools have been developed to help with this. For example, ParaView is a powerful open-source visualization tool that can be used to visualize finite element results. The file format used by ParaView is called *VTK* (Visualization Toolkit), and in paricular the unstructured meshes of finite elements are stored in either the `.vtu` or `.vtkhdf` formats. Develop functions to export your results so that you can visualize them in ParaView. Demonstrate the results on a simple problem.

Bibliography

Chung, J. and G. M. Hulbert (1993). "A Time Integration Algorithm for Structural Dynamics With Improved Numerical Dissipation: The Generalized-α Method". *Journal of Applied Mechanics*, **60**: pp. 371–375. DOI: 10.1115/1.2900803.

Newmark, N. M. (1959). "A method of computation for structural dynamics". *Journal of the Engineering Mechanics Division ASCE*, **85**:EM 3, pp. 67–94.

Witherden, F. and P. Vincent (2015). "On the identification of symmetric quadrature rules for finite element methods". *Computers & Mathematics with Applications*, **69**:10, pp. 1232–1241. DOI: 10.1016/j.camwa.2015.03.017.